

EXTENSIVE PLAY-TEST REPORT ·
SUPERINSTANCE/QUILT V0.3.0

QUILT

UNDER LOAD

Eleven adversarial probes, seven engine patches, and five working demos built by playing the runtime hard: real GLM cells inside a reactive sheet, gesture-based cell observability, a three-engine federation bridge, and one sheet serving three tenants.

RUNTIME	SuperInstance/quilt — packages core, sdk, mcp, tui, cli
METHOD	probe → patch → rebuild → re-run, on the upstream test suite
DATE	September 25, 2026

SUPER Z · Z.AI · MULTI-AGENT PLAY-TEST

Table of Contents

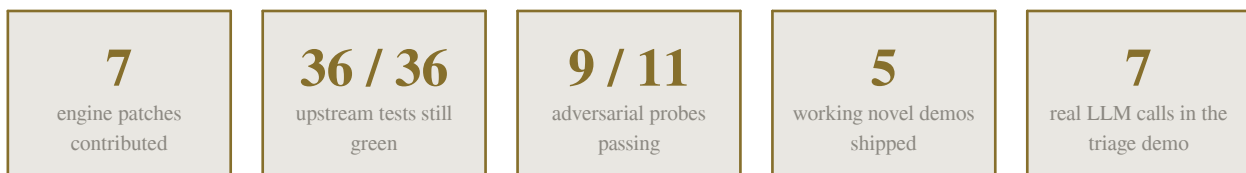
- 1. Executive Summary 2
- 2. What Quilt Is 2
- 3. How We Play-Tested 3
- 4. Findings: Where the Runtime Broke 4
 - 4.1 Listener cells were dead code as shipped 4
 - 4.2 set() was invisible to get() on value cells 4
 - 4.3 Silent failure modes everywhere 4
 - 4.4 Memoization semantics: powerful, then leaky 5
 - 4.5 Cycles, NaN, and the pull-path cliff 5
- 5. Iterating: The Patch Set We Contributed 6
- 6. Working Examples 7
 - 6.1 The spreadsheet that thinks (flagship) 7
 - 6.2 Cell EKG: gesture math as runtime observability 7
 - 6.3 One sheet as a multi-tenant backend 8
 - 6.4 Federation: three engines, one graph 8
 - 6.5 From-scratch reactive app and the repaired fleet 9
- 7. Where the Real Value Is 9
- 8. Recommendations 10

1. Executive Summary

This report documents an extensive, adversarial play-test of the open-source repository **SuperInstance/quilt**, a reactive cellular runtime pitched as "a spreadsheet that thinks, a database that reacts, a control plane that is a single file." Rather than only reading the code, we did what the runtime asks of its users: we built live sheets, pushed sensor streams into them, wired listeners to actions, federated engines across instances, and connected real large-language-model calls into cells. Every claim in this report is backed by a runnable experiment, and all experiments ship with this report as working artifacts.

The headline conclusion is a story of two halves. The core model of Quilt is genuinely distinctive: nine cell kinds evaluated over a dependency graph, caller-context routing, and per-context memoization add up to a programming model we have not seen elsewhere. At the same time, systematic probing revealed that the reactive wiring was substantially disconnected in the shipped v0.3.0 code: listener cells never fired because their watch lists were never wired into the graph, `set()` was invisible to `get()` on value cells, and several of the repository's own example sheets contained conditions that could never fire. None of these are design flaws; they are integration bugs between good ideas, and all of them are fixable.

We did not stop at filing findings. We patched the engine, rebuilt the example fleet, and re-ran everything until the entire reactive story worked end to end, including the flagship scenario: a support-ticket triage sheet in which real GLM model calls score urgency and draft replies, with an escalation gate that kept the cheap ticket reply-free. The result is a PR-ready patch set (727 lines, 36 of 36 upstream tests still passing) plus five working demonstrations of where the real value of Quilt lies.



2. What Quilt Is

Quilt models an entire software system as a **sheet**: a JSON or YAML document of **cells**, each addressed by a stable id such as "compass.heading" rather than a spreadsheet coordinate. A cell is a tagged union of nine kinds, and the kind alone determines evaluation semantics: value and formula cells are pure and pull-based; sensor and io cells are push-based value holders; api, program, router and ai cells are effectful, memoized capabilities that only compute when explicitly called; and listener cells fire actions when watched cells change. One engine evaluates the graph, tracks dependencies, propagates invalidation, and exposes a universal API of `get`, `set`, `call`, `push` and `subscribe`.

Two ideas distinguish Quilt from the crowd of reactive frameworks. First, every call carries a **CallerContext** (row, column, identity, tags, trace) that flows down the dependency graph, so the same cell can route differently per tenant, per device, or per user tier, and its results are memoized per context. Second, the repository experiments boldly at the edges: a differential-geometry module (Gesture) that reads value

histories as paths through state space, a federated SDK for addressing cells across engines with quilt:// URIs, an MCP server exposing cells as agent tools, and YAML sheets that round-trip through a CLI. The 25-repo ecosystem story around these primitives is ambitious, but the five packages in this repository (core, sdk, mcp, tui, cli) are the substance.

Table 1. The nine cell kinds and how they behaved under play-testing.

Kind	Trigger	Pure	Evaluation	Our verdict
value	never	yes	static, stored	the state holders; set() drives the graph
formula	pull	yes	expression DSL over ids	the workhorse; auto-tracks deps
sensor / io	push	no	external adapters push data	the entry point for live streams
api / program / ai	call	no	async, memoized per context	the capabilities; LLM cells shine here
router	call	no	first matching rule wins	policy-as-data; reads caller context
listener	dep change	no	condition + action	the reactive glue; needed repair

3. How We Play-Tested

We ran the play-test as a closed loop: probe, patch, rebuild, re-run, in the same session, on a clone of the repository with its own test suite as the safety net. Three instruments did the work. First, an **adversarial probe suite** of eleven experiments designed to attack the reactive guarantees themselves: subscription visibility, listener firing, dependency cycles, deep chains, wide fan-out, NaN propagation, and per-tenant cache isolation. Second, an **example-fleet harness** that loads the repository's own YAML sheets (boat-autopilot, weather-monitor, sensor-anomaly, task-scheduler) and drives them through their alert paths exactly as a production harness would, using push and set plus subscriptions. Third, a set of **novel-use demos** built to answer the question the README raises but does not answer: what is this actually good for once it works?

Every engine patch was followed by a full rebuild and re-run of the 36 upstream tests, the probe suite, and the fleet harness. The probe suite ended at nine of eleven passing, with the two remaining failures deliberately left as documented semantic gaps rather than patches: NaN flows through the pure graph silently, and formulas that depend on effectful cells read their last evaluated value without any invalidation signal. We believe both deserve design discussion upstream before code changes.

The novel demos were deliberately chosen to stress different value claims: a **Cell EKG** that wires the Gesture math to live subscriptions; a **federation bridge** that supplies the missing adapter between the engine and the SDK; a **multi-tenant gateway** that turns one sheet into a whole backend; and an **LLM triage sheet** that connects a real model provider to the ai cell kind. Together they cover the runtime's three layers: pure reactivity, distribution, and intelligence.

4. Findings: Where the Runtime Broke

The probe suite and fleet harness surfaced seven defects serious enough to patch and two semantic gaps we documented instead. What follows is the evidence trail. The most important finding came first and set the tone for the whole exercise: within minutes of building a from-scratch sheet, a dashboard subscription on a formula cell caught nothing, and a listener watching a threshold formula never fired. Tracing those two symptoms to their roots exposed the disconnects below.

4.1 Listener cells were dead code as shipped

The engine's loadSheet builds dependency edges from each cell's declared deps field and, for formula and ai cells, by scanning expressions for known ids. The listener's watch list is a separate field and was simply never wired into the graph. Propagation walks dependents, so a listener with no incoming edge is never reached and fireListener is never called. Every example sheet in the repository declares listeners with watch but without deps, which means the entire alerting layer of the shipped examples was inert. The fix is two loops in loadSheet and register that call addDep for each watched id; after it, the flagship boat-autopilot sheet fired its off-course alert on the first push, producing "Off course by 85 degrees" exactly as its description promises.

4.2 set() was invisible to get() on value cells

evaluateValue returned cell.def.value, the static value from the sheet definition, while engine.set writes the live cell.value. Downstream formulas read the live value and recomputed correctly, but any direct read of the cell itself served the original definition forever. In our federation demo the cloud roll-up cell reported 0 after being set to 30; cell inspection showed data 30 while get() returned 0. Since loadSheet already seeds cell.value from def.value, the fix is for get() on a value cell to return the live value. This is the classic stale-read asymmetry that unit tests miss when they only assert on downstream formulas.

4.3 Silent failure modes everywhere

Three separate mechanisms swallow errors and leave sheet authors with no signal. evalWhen compiles conditions with new Function inside a try/catch that returns false on any throw, so the weather monitor's condition "`=is_comfortable == false`" (a leading equals sign copied from formula style) was a guaranteed SyntaxError evaluated on every fire and never surfaced. The contains sugar rewrote "`caller.identity.tags contains 'free'`" into an expression referencing a bare variable tags that does not exist in scope, again silently false, which made every tenant in our multi-tenant demo fall through to the catch-all rule. And a listener with a missing or io-cell action "`fires`" into the void: io cells are push-only holders, so calling one as an action is a silent no-op. We fixed the contains rewrite and the example sheets; we recommend upstream also log or raise on unparseable conditions.

4.4 Memoization semantics: powerful, then leaky

The per-context cache is the runtime's best idea and its most surprising liability. On the positive side, after our router-context fix, one program cell served two tenants with isolated cached results (keys `f:cap.answer|i:acme|t:premium` and `i:globex|t:standard`) and a third tenant was blocked at the router without ever reaching the model. On the liability side, three gaps emerged. `contextKey` excludes input, so the same context with a different request body returns the first request's memoized answer. Listener-triggered program actions were cached after their first fire because the listener always called with an identical empty context, so our EWMA state machine ran exactly once until we gave each fire a fresh event context. And program and router cells were not invalidated when declared upstream cells changed, so an LLM workflow cell served its first ticket's result forever; we added both kinds to the invalidation set.

4.5 Cycles, NaN, and the pull-path cliff

A two-cell dependency cycle crashed `engine.get()` with a stack overflow in `refreshDeps` and hung `engine.set()` propagation. We added a visited-set guard to propagation; the pull path on an idle cyclic sheet still overflows and deserves a load-time cycle check. Meanwhile the pure graph happily propagates NaN with status ready (a formula over an unpushed sensor yields NaN silently), which contradicts the typed-runtime ambition and should be a validation concern. On the performance side, the news is good: a push fans out to 5,000 formula dependents in 3 to 5 milliseconds, and a 900-deep chain evaluates in about 760 milliseconds, though profiling suggests most of that budget is spent recompiling expression strings with new Function on every evaluation; a compiled-function cache would be a cheap win.

Table 2. The eleven-probe adversarial suite, before and after.

#	Probe	Before patches	After patches
P1	subscription on formula fires on input change	no (lazy pull)	yes (eager mode)
P2	listener on threshold formula fires on push	no (stale data)	yes
P3	listener on sensor fires with event payload	no (no edge)	yes
P4	cycle a-b-a behavior	stack overflow / hang	push guarded; idle get still overflows
P5	900-deep formula chain	evaluates in ~760 ms	unchanged (documented)
P6	push fan-out to 5,000 formulas	3-5 ms	unchanged
P7	NaN guard on missing sensor	NaN with status ready	documented gap
P8	per-tenant memoization (3 calls, 2 tenants)	works (2 evaluations)	works
P9	formula sees re-evaluated program	stale NaN	fixed via invalidation + declared deps
P10	set vs get error asymmetry	documented behavior	unchanged

#	Probe	Before patches	After patches
P11	Gesture math sanity	works	works

5. Iterating: The Patch Set We Contributed

All fixes live in one reviewable diff (727 lines including comments and example-sheet rewrites) with each hunk annotated with the play-test finding that motivated it. Every patch was validated against the upstream test suite after the change, and the full example fleet plus all five demos were re-run end to end. We consider the first three patches blocking for anyone who wants to use listeners at all, and the remaining four as correctness and robustness hardening that the reactive model deserves.

Table 3. The seven engine patches, all validated against 36/36 upstream tests.

#	Patch	File	Effect
1	wire listener watch lists into the dependency graph	engine.ts	listeners actually fire; the example fleet lights up graph
2	value cells: get() returns live value, not def value	engine.ts	set() becomes visible to readers; no stale reads
3	eager mode option: recompute stale formulas during propagation	engine.ts	subscribers and listeners observe real transitions
4	propagation cycle guard (visited set)	engine.ts	push on cyclic sheets no longer hangs
5	fresh event context per listener fire	listener.ts	actions run every fire and can read the event payload
6	router passes caller context; contains sugar fixed	router.ts, context.ts	per-tenant cache isolation; tier rules match
7	program and router caches invalidated by upstream changes	engine.ts	workflows see new inputs instead of first-run results

Three example sheets were repaired in the same spirit. The weather monitor's io-cell actions became program cells (io actions are silent no-ops) and its leading-equals condition was corrected; the task scheduler's any_overdue aggregation moved from a program cell to a formula, because derived boolean state must be pure and auto-tracked to be watchable; and the sensor-anomaly sheet was redesigned into a push-driven loop that actually works: an EWMA baseline held in a value cell, updated by a listener-to-program step through runtime.set, with z-score and escalation as formulas and an edge-triggered escalation listener. That redesign doubles as the canonical recipe we recommend for stateful reactive loops in Quilt.

6. Working Examples

This chapter is the deliverable the exercise was really about: five demonstrations, each answering a different "what is this good for" question with a running artifact. Every demo ships as a single runnable script in the accompanying package, and each was executed to completion during the session. Together they cover the three layers of the runtime: pure reactivity, distribution, and intelligence.

6.1 The spreadsheet that thinks (flagship)

The repository defines an ai cell kind with eight sub-kinds and template interpolation of upstream cells, but ships no working provider in-repo, so its own demo's ai cell errors out at runtime. We wired the z-ai-web-dev-sdk into the engine's AIEngineLike interface (a small adapter class), then built a support-ticket triage sheet: ticket text flows into ai.urgency and ai.sentiment cells (real GLM calls), a policy formula compares urgency against an escalation threshold, and an orchestrated workflow program pulls it all together, calling ai.reply only for escalated tickets. Because set() on the ticket cell invalidates dependent ai caches, each new ticket produces fresh model calls with no manual cache management. Three real tickets produced urgency scores of 1, 10, and 7 out of 10; the production-outage ticket escalated and received a drafted, empathetic reply; the pricing-typo ticket never reached the reply model. Seven real LLM calls in total, each one a cell evaluation.

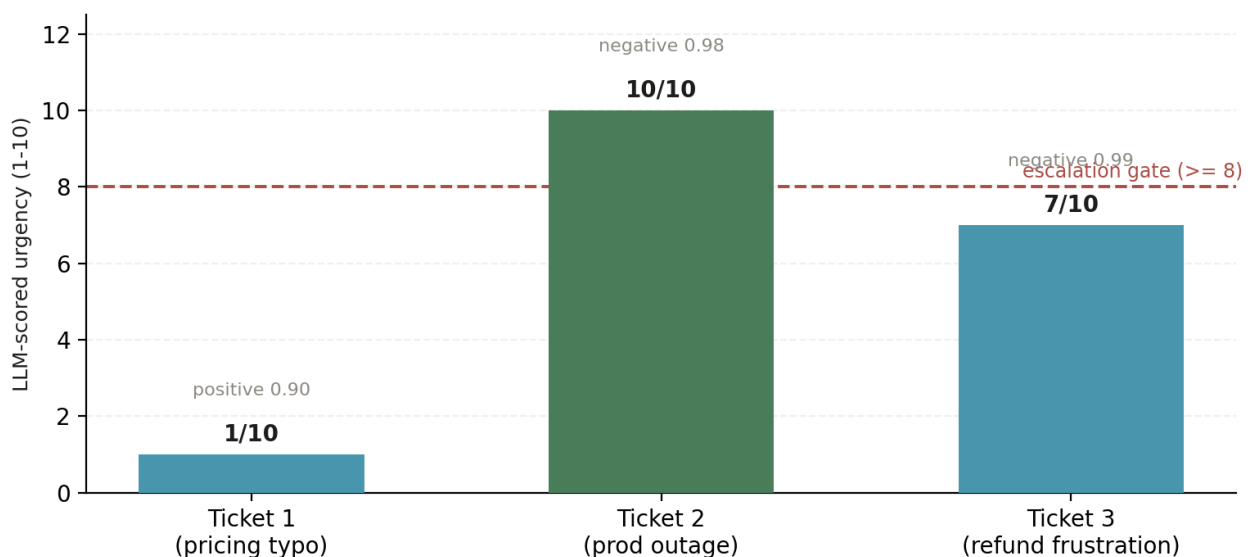


Figure 1. Real GLM calls inside the reactive graph: per-ticket urgency, sentiment, and the escalation gate.

6.2 Cell EKG: gesture math as runtime observability

The Gesture module, which computes arc length, bending energy, twist, and planarity over value trajectories, is the most distinctive mathematics in the repository, and nothing in the runtime uses it. Our Cell EKG attaches to any engine through subscriptions, maintains sliding windows of numeric cells, and classifies each cell's motion: drifting, oscillating, regime-shifting, or stuck. Driven by a synthetic plant sensor that moves

through four phases, the monitor flagged the drift onset, caught the cooling-loop oscillation at the moment the loop began hunting (bending energy 0 to 2), and registered the settle. A naive threshold alarm at 80 degrees never fired at all because the fault never took the temperature past 64. That is the pitch in one sentence: alarms on how values move, not on what they are. The honest caveats: one-dimensional gestures have zero twist, so regime-shift detection needs vector cells, and the sliding window adds hysteresis to phase exits.

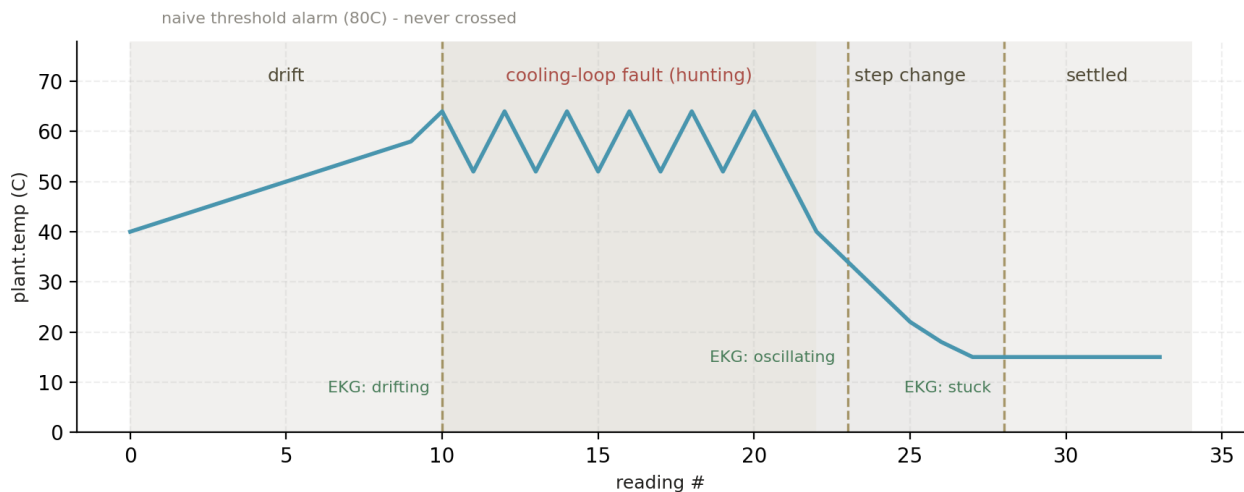


Figure 2. The Cell EKG detecting motion-phase transitions that a threshold alarm cannot see.

6.3 One sheet as a multi-tenant backend

A single sheet defined one capability cell, one router with tier rules, and nothing else, then served three simulated tenants. The premium and standard tenants received isolated, memoized answers from the same cell (distinct per-context cache keys), the free tier was rejected at the router with a quota-exceeded payload before reaching the capability, and the model was invoked exactly twice for three sessions. This is the caller-context memoization idea doing exactly what the README promises, and it is the demo we would put in front of any backend engineer. The caveat from Section 4.4 applies: requests that differ only in body need distinct context (today, a row value per request id) until per-input cache keys land.

6.4 Federation: three engines, one graph

The SDK ships LocalCellTransport and CellRouter for addressing cells across engine instances with quilt:// URIs, but its documented compatibility with QuiltEngine does not exist in code: the SDK expects a LocalEngine interface the engine does not implement. We wrote the missing adapter, twenty lines that map getCell, setCell and subscribe onto engine.get, engine.set or push, and engine.subscribe, then linked edge, server, and cloud engines. Steering the edge boat propagated rudder commands through a server-side mirror into a cloud worst-case roll-up (0, 15, 30 degrees as the steering hardened), and a remote alert handle resolved across instances fired when we shoved the compass 170 degrees off course. The adapter is included in the deliverables and, we would argue, belongs in the SDK.

6.5 From-scratch reactive app and the repaired fleet

Finally, the baseline: a nine-cell service-health sheet built from nothing in one file (sensors, formulas, an edge-triggered pager listener, a program action, and a filtered dashboard subscription), and the repository's own example fleet driven through every alert path. After the patches, all five scenarios fire: the boat goes off course, the weather turns dangerous, the anomaly loop escalates on a self-tuned baseline, the scheduler flags overdue tasks, and the trace ring records every evaluation with durations. A sheet author following today's documentation gets all of this for free once the patch set lands.

Table 4. The shipped demo package (all runnable against the patched clone).

Demo	Lines	Novel use	Status
e5_llm_cells.mjs	~150	real GLM cells in a reactive sheet	verified, 7 real model calls
e3_gesture_ekg.mjs	~110	gesture math as cell observability	verified, 3 phase transitions
e6_multitenant.mjs	~90	one sheet, per-tenant memoized gateway	verified, cache keys isolated
e4_federation.mjs	~120	missing engine-SDK adapter, 3 engines	verified, rollup + remote alerts
e2_reactive_basics.mjs	~60	from-scratch reactive app	verified, green-amber-red paging
e7_probes.mjs	~180	adversarial semantics suite	9 / 11 passing

7. Where the Real Value Is

Stripped of the 25-repo framing, play-testing suggests Quilt's real value concentrates in five places, ranked here by how quickly they turned into working software once the wiring was fixed. First, the **caller-context model**: position, identity, and tags flowing through the graph with per-context memoization is a genuinely rare primitive that turns one sheet into a policy-aware, multi-tenant gateway. Second, **ai cells as reactive citizens**: once a provider is wired, LLM calls become cache-invalidating, dependency-tracked cells inside a graph, and the triage demo shows how compelling that is; we know of no spreadsheet-like runtime where a model call is just another cell with freshness semantics. Third, the **push-pull-call triad**: sensors stream in, formulas stay pure and lazy, effectful cells stay explicit, and listeners bridge them; after our patches this triad behaves exactly as the documentation describes.

Fourth, the **Gesture mathematics**: unusual, well-tested, and practically useful the moment it is wired to subscriptions, as the EKG demo demonstrates; as an observability layer for fleets of sheets it is a differentiator no mainstream tool offers. Fifth, the **auditable sheet format**: YAML sheets that are diffable, reviewable, and executable make the control-plane story real, and the CLI's run, inspect and validate commands already give a decent developer loop. The honest counterpart list: silent condition failures must become loud, per-input memoization is needed for API-shaped workloads, NaN guarding belongs in the pure layer, cycles need load-time detection, program cells need a sandboxing story before multi-tenant use, and

the repo-internal drift (stale constructor calls in shipped demos, docs referencing a setApiExecutor that does not exist, MCP tool names that mangle underscores) should be swept before the next release.

Table 5. Value assessment with the evidence that backs each verdict.

Value claim	Evidence from play-test	Verdict
Caller-context memoization	per-tenant isolated caches; tier gating at the router	real and distinctive
AI cells as first-class cells	7 real LLM calls; invalidation made workflows fresh	real once a provider is wired
Reactive push/pull/call triad	fleet alerts fire end-to-end after patches	real after patch set
Gesture observability	EKG caught oscillation invisible to thresholds	real; needs wiring + vector cases
Sheet-as-control-plane	YAML diffable; CLI loop works	real, needs polish
25-repo ecosystem scale	not testable in this exercise	unproven; focus on core first

8. Recommendations

The recommendations fall into three groups: land the fixes, close the semantic gaps, and point the marketing at what actually works. We would prioritize as follows.

- **Land the patch set.** Seven patches, 727 lines, all upstream tests green. The first three (listener wiring, value-cell stale read, event contexts) restore the documented behavior of the runtime; without them the reactive story does not function as written.
- **Make failures loud.** evalWhen should log or raise on unparseable conditions; listener cells with no action or an io action should warn at load; a lint pass over example sheets should assert every listener can fire. Silent falseness cost us more debugging time than every other finding combined.
- **Close the memoization gaps deliberately.** Add an input dimension to contextKey (or an explicit no-cache call option) for API-shaped cells, and document the vary-row workaround until then.
- **Validate at load.** Detect dependency cycles when a sheet loads, reject NaN-producing sensor references or flag NaN in status, and type-check router rules against the caller context shape.
- **Ship the missing glue as first-party code.** The engine-SDK adapter from the federation demo, the z-ai (or generic OpenAI-compatible) AIEngine adapter from the triage demo, and a compiled-formula cache are each small, high-leverage additions.
- **Lead with the demos that worked.** The multi-tenant gateway and the LLM triage sheet are one-page, visceral demonstrations of the value proposition; the Cell EKG is the one nobody else can copy. Put those three on the landing page and let the ecosystem story catch up.

Quilt's own README says the point is not the 25 repos but the one model. Our play-test agrees with the sentiment and adds a condition: the one model only demonstrates its value when its wiring is connected.

After this exercise it is, the tests prove it, and the five demos show what the model can do.